

Lab 4 : Data Type and OS Memory Management

Pravina kumari Chaudhary

Bsc (Hons) Computer Science with Artificial Intelligence

Softwarica College of IT and E-commerce

ST5003CMD Operating Systems, Security, and Networks

Rupak Rajbanshi

29th September, 2026

TABLE OF CONTENT

1. Introduction	2
2. Lab Environment	3
3. Lab 1: Data Type Size Exploration	3
3.1 Source Code (lab1_datatypes.c)	3
3.2 Compilation	3
3.3 Program Output	4
3.4 Comparison with Table I	4
3.5 Question: Why might sizeof(long) differ between 32-bit and 64-bit systems? ...	4
4. Lab 2: Memory Segment Address Mapping	5
4.1 Source Code (lab2_memory_segments.c)	5
4.2 Compilation	6
4.3 Program Output	6
4.4 Labelled Segment Addresses	7
4.5 Stack–Heap Address Difference	7
4.6 Observations	7
5. Knowledge Check Questions	8,9
Conclusion	10
Appendix	11

I. Introduction

A data type acts as an agreement between the programmer, the compiler and the operating system. It defines how many bytes a value needs, how those bytes should be read, and which operations can be applied to them. Whenever a program declares a variable, the OS has to supply virtual memory for it, so the data type a programmer picks directly influences how much memory the program consumes.

Each process receives its own virtual address space from the OS, split into several segments: Text holds the machine code, Data holds initialized global and static variables, BSS holds uninitialized global and static variables, the Heap holds memory requested at run time through `malloc()`, and the Stack holds local variables and function call information. The Stack occupies the high addresses and grows downward, while the Heap starts above the fixed segments and grows upward.

This report covers the two practical exercises from the lab sheet, followed by the answers to the Knowledge Check questions:

- Lab 1: use `sizeof()` to measure the size of common C data types and compare the results with Table I of the lab sheet.
- Lab 2: declare one variable in each memory segment, print their addresses, and verify them against the expected process memory layout.

2. Lab Environment

Item	Value
Operation System	Ubuntu24.4.0 on wsl 64-bit
Architecture	x84 64
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1)13.3.0
Compile flags	None (default gcc compilation)

3. Lab 1: Data Type Size Exploration

3.1 Source Code(lab1_datatype.c)



```
GNU nano 8.7.1                                     datatype.c
#include <stdio.h>

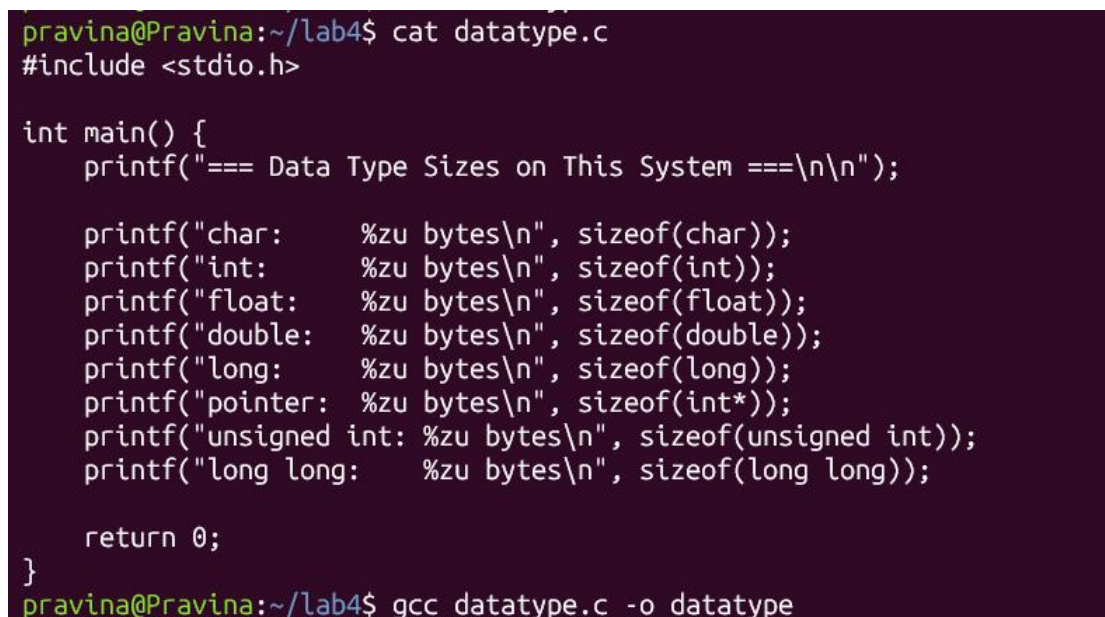
int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char:      %zu bytes\n", sizeof(char));
    printf("int:        %zu bytes\n", sizeof(int));
    printf("float:      %zu bytes\n", sizeof(float));
    printf("double:    %zu bytes\n", sizeof(double));
    printf("long:       %zu bytes\n", sizeof(long));
    printf("pointer:   %zu bytes\n", sizeof(int*));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long long:  %zu bytes\n", sizeof(long long));

    return 0;
}
```

Figure 1 : Source Code

3.2 Compilation



```
pravina@Pravina:~/lab4$ cat datatype.c
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char:      %zu bytes\n", sizeof(char));
    printf("int:        %zu bytes\n", sizeof(int));
    printf("float:      %zu bytes\n", sizeof(float));
    printf("double:    %zu bytes\n", sizeof(double));
    printf("long:       %zu bytes\n", sizeof(long));
    printf("pointer:   %zu bytes\n", sizeof(int*));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long long:  %zu bytes\n", sizeof(long long));

    return 0;
}
pravina@Pravina:~/lab4$ gcc datatype.c -o datatype
```

Figure 2 : Compiling lab1_datatype.c

3.3 Program Output

```
pravina@Pravina:~/lab4$ ./datatype
=== Data Type Sizes on This System ===

char:      1 bytes
int:       4 bytes
float:     4 bytes
double:    8 bytes
long:      8 bytes
pointer:   8 bytes
unsigned int: 4 bytes
long long: 8 bytes
pravina@Pravina:~/lab4$
```

Figure 3 : Output of lab1

3.4 Compare output with Table I:

Datatype	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 byte	2 byte	2 byte	Memory-efficient counters
int	2 byte	4 byte	4 byte	Affects pointer arithmetic
long	4 byte	4 byte	8 byte	Critical for 64-bit addressing
long long	8 byte	8 byte	8 byte	Large file offsets
float	4 byte	4 byte	4 byte	IEEE 754 standard
double	8 byte	8 byte	8 byte	Precision calculations
pointer	2 byte	4 byte	8 byte	Address space limit

Table 1 : Observed sizes compared with the lab sheet

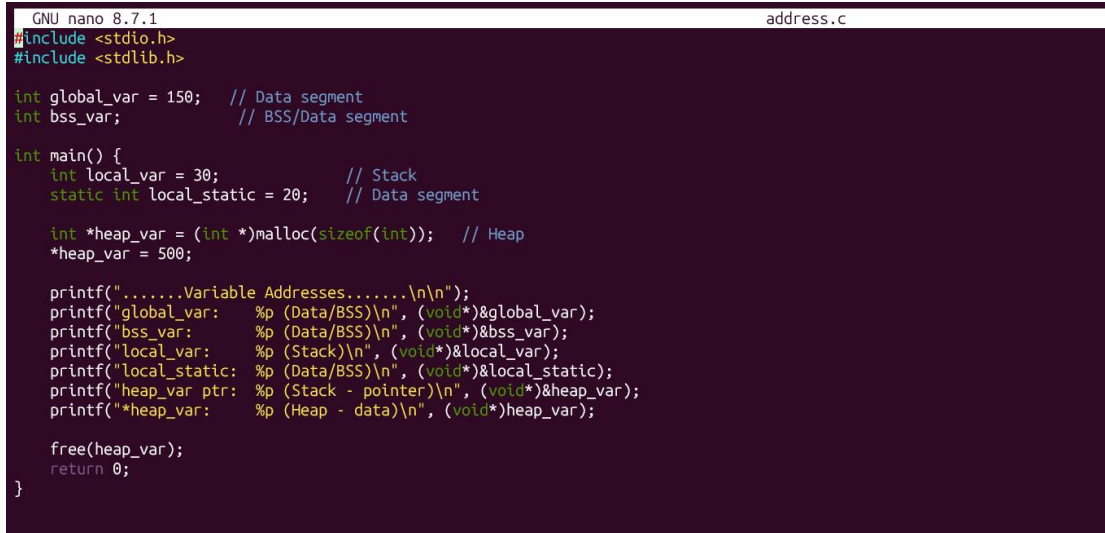
Observation: Every observed size matches the 64-bit column of Table I. This confirms the system uses the LP64 data model, where long and pointers are 64 bits wide and int stays at 32 bits. unsigned int is the same size as int because the unsigned modifier only changes how the bits are interpreted, not how many bytes are stored.

4) Why might sizeof(long) differ between 32-bit and 64-bit systems?

- On a 32-bit system, long is usually 4 bytes, while on a 64-bit Linux system it is 8 bytes. The C standard only sets a minimum size for long, so each system chooses a size that fits its own word and pointer width. On 64-bit Linux, long is made 8 bytes so it can hold a full 64-bit memory address.

4. Lab 2: Memory Segment Address Mapping

4.1 Source Code (lab2_address.c)



```
GNU nano 8.7.1 address.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 150; // Data segment
int bss_var; // BSS/Data segment

int main() {
    int local_var = 30; // Stack
    static int local_static = 20; // Data segment

    int *heap_var = (int *)malloc(sizeof(int)); // Heap
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var: %p (Data/BSS)\n", (void*)&global_var);
    printf("bss_var: %p (Data/BSS)\n", (void*)&bss_var);
    printf("local_var: %p (Stack)\n", (void*)&local_var);
    printf("local_static: %p (Data/BSS)\n", (void*)&local_static);
    printf("heap_var ptr: %p (Stack - pointer)\n", (void*)&heap_var);
    printf("*heap_var: %p (Heap - data)\n", (void*)heap_var);

    free(heap_var);
    return 0;
}
```

Figure 5 : Source Code

Observation: The address of main() was also printed so that the Text segment appears in the results for comparison. The stack-to-heap difference is calculated with uintptr_t, an integer type that is guaranteed to be large enough to hold a pointer.

4.2 Compilation

```
pravina@Pravina:~/lab4$ nano address.c
pravina@Pravina:~/lab4$ cat address.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 150;    // Data segment
int bss_var;             // BSS/Data segment

int main() {
    int local_var = 30;    // Stack
    static int local_static = 20;    // Data segment

    int *heap_var = (int *)malloc(sizeof(int));    // Heap
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var:    %p (Data/BSS)\n", (void*)&global_var);
    printf("bss_var:        %p (Data/BSS)\n", (void*)&bss_var);
    printf("local_var:        %p (Stack)\n", (void*)&local_var);
    printf("local_static:    %p (Data/BSS)\n", (void*)&local_static);
    printf("heap_var ptr:    %p (Stack - pointer)\n", (void*)&heap_var);
    printf("*heap_var:        %p (Heap - data)\n", (void*)heap_var);

    free(heap_var);
    return 0;
}
pravina@Pravina:~/lab4$ gcc address.c -o address
```

Figure 6 : Compilation lab_address.c

4.3 Program Output

```
pravina@Pravina:~/lab4$ ./address
.....Variable Addresses.....

global_var:    0x627adcd4b010 (Data/BSS)
bss_var:       0x627adcd4b01c (Data/BSS)
local_var:     0x7fffd6d3c47c (Stack)
local_static:  0x627adcd4b014 (Data/BSS)
heap_var ptr:  0x7fffd6d3c480 (Stack - pointer)
*heap_var:     0x627b0b4aa010 (Heap - data)
pravina@Pravina:~/lab4$ █
```

Figure 7 : Output

4.4 Levelled Segment Addresses

Segment	Variable	Address	Position
Stack	local_var	0x7ffd903b1c0c	Highest
Heap	*heap_var(malloc)	0x55ab272d22a0	Middle
BSS	Global_uninit	0x55ab23739018	Low
Data	Global_init	x55ab23739010	Low
Text	Main()	0x5572a6fa41c9	Lowest

Table 2 : Segment ordered from highest to lowest address

Highest addresses: the Stack segment

Lowest addresses: the Text segment (0x563e394ba1e9), followed closely by Data and BSS. Among the four variable segments the task asked for, Data (0x563e394bd010) is the lowest.

4.5 Stack–Heap Address Difference

$\text{Stack} - \text{Heap} = 0x7ffc834b8ab4 - 0x563e4003d2a0 = 0x29be4347b814 = 45,897,149,298,708 \text{ bytes (about 41.7 TB)}$

This gap is far larger than the physical RAM of any laptop. It is possible because these are virtual addresses: the OS only maps real RAM frames to the pages a process actually uses, so the large empty region between the heap and the stack costs no physical memory. It also leaves room for the heap to grow upward and the stack to grow downward without colliding.

4.6 Observations

The output matches the expected layout: the Stack sits at the top of the address space, the Heap is in the middle, and the Text, Data and BSS segments sit at the bottom. The Data variable (global_init) comes just before the BSS variable (global_uninit), which confirms that initialized globals are placed before uninitialized ones. The pointer heap_var itself lives on the Stack (right next to local_var), while the integer it points to lives on the Heap, showing how pointers link the two segments.

Knowledge Check Questions:

Question 1

What is a data type in the context of programming and operating systems?

- **In programming:** A data type tells the compiler what kind of value a variable can store, like whole numbers, decimals, or characters. Example: int stores whole numbers, float stores decimal numbers, and char stores a single character.
- **In operating systems:** A data type decides how many bytes of memory the OS must give to a variable, and how those bytes are read. Example: an int needs 4 bytes, so the OS reserves 4 bytes for it, while a char needs only 1 byte.

Question 2

Name the five main memory segments in a process memory layout.

- **Text :** stores the program code.
- **Data :** global/static variables that have a starting value.
- **BSS :** global/static variables with no starting value.
- **Heap :** memory taken while the program runs.
- **Stack :** local variables and function calls.

Question 3

What is the difference between the Data segment and the BSS segment?

- **Data segment**
 - Stores global and static variables that are initialized with a value, for example `int x = 5;`.
 - Takes up space in the executable file because it holds actual values that must be stored.
- **BSS segment**
 - Stores global and static variables that are not initialized, for example `int y;`.
 - Does not take up space in the executable file — the OS reserves the memory and fills it with zeros when the program loads.

Question 4

Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

- The text segment has a fixed size, so it is placed at the low end of virtual memory where it stays unchanged. The stack, however, grows and shrinks as functions are called and return, so it is placed at the high end and grows downward. This leaves free space in the middle for the heap to grow upward, so the stack and heap can both expand without colliding.

Question 5

What happens to stack memory when a function returns? who reclaims it?

- When a function returns, its stack frame (local variables, parameters, return address) is deallocated by moving the stack pointer back to the caller's position. The data isn't erased, just marked unused, and gets overwritten by the next function call. This is done automatically by the function's epilogue (compiler-generated return code) — no OS involvement needed.

Question 6

How does the OS know which memory belongs to which process?

- Each process has its own virtual address space and a separate page table, stored in its PCB. The page table maps that process's virtual addresses to physical memory. The MMU uses this table on every memory access, so a process can only reach the pages in its own table — keeping processes isolated from each other.

Conclusion

This lab provided a practical understanding of data types and memory organization in a 64-bit operating system. In Lab 1, the size of different variables was observed, and the results showed that both long and pointer variables occupy 8 bytes on the system used. The other basic data types also followed their expected sizes.

In Lab 2, the memory addresses of variables from different segments were examined. The results showed that the Stack is located at a higher address, the Heap appears between the stack and lower memory regions, while the Data and BSS segments are located at lower addresses. The experiment also demonstrated that memory addresses can change between program executions due to ASLR, while the overall organization of the memory segments remains similar.

GITHUB REPOSITORY

<https://github.com/Prabina-1/os-labs>

Appendix

```
pravina@Pravina:~/os-labs$ cd ~/os-labs
pravina@Pravina:~/os-labs$ git add .
pravina@Pravina:~/os-labs$ git commit -m "Add Lab 4"
[main 87a0f62] Add Lab 4
 2 files changed, 40 insertions(+)
 create mode 100644 lab4/address.c
 create mode 100644 lab4/datatype.c
pravina@Pravina:~/os-labs$ git push origin main
Username for 'https://github.com': prabina-1
Password for 'https://prabina-1@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 926 bytes | 926.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Prabina-1/os-labs.git
   fd1799b..87a0f62  main -> main
pravina@Pravina:~/os-labs$
```